

Golden Thread Plugin

Golden Thread Plugin — Documentation

Version gt 0.12.3 / gt-wiki 0.1.2

Golden Thread turns an Obsidian vault into the single source of truth for all AI memory across every project and every session. The tiered rule model introduced in v0.6.0 now carries **seven hook-backed Core rules** enforced at three points in the turn, 0.9.12 added `gt-route` for the middle of a session, and 0.9.13 makes the session-start component check verify that the hooks are **wired**, not merely installed, 0.11.0 makes `log.md` and `decisions.md` generated files so concurrent sessions cannot overwrite one another, and 0.12.3 stops a vault tool running against a vault it was never told to touch. gt-wiki 0.1.2 provides an LLM-powered knowledge base with immutable sources and interlinked pages.

What It Does

Instead of scattered `.claude/memory/` files and CLAUDE.md snippets that live and die per session, Golden Thread gives every fact a permanent home in a structured vault. Knowledge flows up a hierarchy from session notes into the vault, and the right facts are always in scope when you need them.

```
session conversation
  ↓ gt-work / gt-promote
project memory (memory/*.md)
  ↓ gt-promote
project files (research.md / decisions.md / design.md / spec.md / runbook.md)
  ↓ gt-promote
Knowledge/      (cross-project wiki pages, backed by immutable Sources/)
```

```
↓ gt-promote
global-memory/ (loaded in every session, all projects)
```

Facts move up the hierarchy as they prove themselves general. They never move back down and are never silently deleted.

Core Rules (gt 0.12.3)

Golden Thread defines a tiered rule model that separates rules by scope and enforcement strength. **Seven Core rules ship as of 0.12.3**, up from one at 0.6.0:

#	Rule
1	Register your session and claim a vault file before writing it; never write a file another live session has claimed.
2	Never put a secret's value into the session — not to inspect it, not to redact it, not to check it.
3	Begin every response with the current wall-clock timestamp — before any other text you emit.
4	<code>global-memory/</code> contains only facts needed in EVERY project.
5	Do not auto-load the full memory index.
6	A secret's value rests only in the secrets store or a mode-600 file the store wrote — never in source, a vault file, a repo, a log, or a session.
7	Label every derived figure you present as fact with its verification state — <code>unverified</code> , <code>self-verified</code> , OR <code>independently verified</code> .

Verify the set at any time with `echo '{}' | ~/.claude/golden-thread/hooks/inject_core_rules.sh`.

Three scope levels:

Level	Meaning
Core	Applies to every chat, every project, no exceptions. Un-removable.
Context	Applies to this project or session type only.
Generic	Default; no special standing.

Two enforcement strengths:

Strength	Meaning
Reminder	Injected on every turn via <code>UserPromptSubmit</code> hook.
Validated	Checked by a <code>stop</code> hook that blocks any reply violating it; machine-enforced.

Since 0.12.3 a **second `PreToolUse` guard** (`guard_vault_writes.sh`) denies a vault-mutating tool run that does not say *which* vault it means — no `--vault` , no `--dry-run` , no `GT_VAULT` . It exists because a session rehearsing the 0.11.0 migration in a scratch copy had one tool that took `--vault` and one that did not, so the same loop migrated all 42 `decisions.md` files of the live vault. A rehearsal that cannot be told where to rehearse is not a rehearsal. Read-only subcommands are never denied, and anything it cannot parse with certainty is allowed — it fails open, as every guard here does, because a guard that blocks wrongly makes every session unusable.

Since 0.9.5 a third point exists: a **`PreToolUse` hook** (`guard_session_claims.sh`) denies a `Write / Edit` to a vault file another live session holds a claim on. It fails open on any parse failure, and deliberately does not cover `Bash` writes — parsing arbitrary shell fails open so often it would train the guard to be ignored.

Core/Validated rules are hook-backed. Hook scripts install to `~/.claude/golden-thread/hooks/` during `install.sh` . `settings.json` references them by absolute path so they survive vault renames and project moves. `gt_paths.py` is a self-healing resolver — if a recorded path is stale, it locates `core-rules/` by scanning the vault rather than failing silently.

The canonical rule definitions live in `Projects/golden-thread/core-rules/` inside the vault. Editing a rule file there changes what the hook injects — the rule text is never duplicated into the script.

The canary: `core_timestamp_every_message.md` is designated Core/Validated not because a missing timestamp causes real harm, but because it is trivially observable at the top of every reply. When the timestamp disappears, enforcement has broken.

gt Skills (17)

Setup

Command	What it does
<code>/gt:gt-init</code>	Create a new vault from scratch, or wire an existing vault to a project. Idempotent — safe to re-run.
<code>/gt:gt-create</code>	Scaffold a new project folder with the standard structure. Captures your brain dump into <code>idea.md</code> . Supports sub-projects, tags, and runbook creation.

Daily Work

Command	What it does
<code>/gt:gt-open</code>	Load a project at session start. Reads all project docs in order (idea → research → decisions → design → spec → runbook → memory), summarizes state, and asks where to pick up.
<code>/gt:gt-route</code>	Mid-session. Names what the session has actually become, says where its output belongs, and checks you are in the right project, harness and model. For when a session drifted from what it opened with, or you cannot name what you are doing.
<code>/gt:gt-work</code>	Write back session findings. Appends to <code>research.md</code> , adds ADRs to <code>decisions.md</code> , refines <code>design.md</code> , creates <code>spec.md</code> when design is complete, and flags content for <code>PROTOCOL.md</code> .
<code>/gt:gt-ingest</code>	Bulk-import an existing project's memory files, CLAUDE.md rules, and notes into the vault. External sources are stored immutably in <code>Sources/</code> before being synthesized into Knowledge pages.
<code>/gt:gt-review</code>	Scan recent Obsidian daily notes for uncaptured tasks and ideas. Surfaces them grouped by date, then promotes selected ones into tracked project folders.

Knowledge Management

Command	What it does
<code>/gt:gt-query</code>	Look something up — reads <code>index.md</code> first, follows wikilinks, falls back to grep. The vault is always checked before the web.
<code>/gt:gt-promote</code>	Graduate a fact up the hierarchy: project memory → project files → Knowledge wiki page → global-memory. Also handles new project scaffolding and retiring stale content.
<code>/gt:gt-refresh</code>	Check <code>Sources/</code> for upstream changes. Supersedes outdated sources with new immutable files — never edits the old one. Updates Knowledge pages that cited the changed source.

Context & Verification

Command	What it does
<code>/gt:gt-farm</code>	Route bulk, mechanical, or second-opinion tasks to an external AI service as a self-contained work packet. All four gates (Stateless, Self-contained, Checkable, Releasable) must pass before a task leaves. Results come back unverified.
<code>/gt:gt-validate</code>	Verify a claim by re-deriving it with a fresh-context validator — never by reviewing the reasoning that produced it. Use before recording a finding as fact or before a production change.

Maintenance

Command	What it does
<code>/gt:gt-upgrade</code>	Bring an existing vault up to the installed release: run the migrations it has not had, take the release's changes into <code>PROTOCOL.md</code> and <code>CONVENTIONS.md</code> without losing local edits, add newly shipped Core rules, stamp the vault. Rehearse with <code>--dry-run</code> ; it refuses a dirty tree and backs up before applying.

Command	What it does
<code>/gt:gt-doctor</code>	One report for the whole install: plugin version, component drift, hook wiring, pending vault migrations, stray workers, unpushed commits, publish-destination drift and a lint summary. Exit 2 means a check <i>could not run</i> , which is deliberately distinct from clean.
<code>/gt:gt-lint</code>	Audit the vault for structural problems: 14 checks covering broken wikilinks, orphaned pages, missing index entries, unlisted memory files, Knowledge pages citing superseded sources, stale pages, and <code>core-unenforced</code> — a Core rule that is stored but wired to no hook.
<code>/gt:gt-runbook-lint</code>	Scan all project <code>runbook.md</code> files for content that has drifted into multiple runbooks. Routes duplicated content to the right shared layer via <code>gt-promote</code> .
<code>/gt:gt-settings</code>	View and change what Golden Thread does on its own: component drift checking at session start, and the session report card at compact. Every automatic behaviour can be switched off.

Watching upstream

Command	What it does
<code>/gt:gt-watch</code>	Watch any git repo. <code>add <url></code> writes a watch note; a cron fetch (<code>gt_watch.py fetch</code>) classifies every change by rules — P0 for security advisories, CVE/GHSA ids or security releases; review for new releases, major bumps, changes under <code>watch_paths</code> ; routine otherwise — and a SessionStart hook reports unacknowledged changes, P0 first. <code>show</code> explains a change, <code>ack</code> marks it seen. Setting <code>watch</code> , default <code>off</code> .

Demo

Command	What it does
<code>/gt:gt-demo</code>	An eleven-act guided tour on the fictional PizzaBot 3000 project, run in its own throwaway vault — <code>start</code> builds it, <code>tour</code> runs the acts one click at a time (Core rules, open, task rollup, capture, wiki, route, validate, lint, promote, watch,

Command	What it does
	inbox and close), <code>end</code> shows the receipt, <code>clean</code> rebuilds, <code>remove</code> switches it off. Skipped at install when <code>install_demo</code> is <code>no</code> .

Concurrent sessions stop colliding (0.12.3)

`log.md` and `decisions.md` are the two files every session appends and none owns. In one working tree there are no branches to collide and no merge to resolve — just last-writer-wins, invisible when it happens.

Both are now **generated**, joining `TASKS.md` :

File	Written by	Rendered by
<code>log.md</code>	<code>gt_log.py add</code> — one spool file per session	<code>gt_log.py merge</code>
<code>decisions.md</code>	<code>gt_adr.py allocate <project></code> — one file per ADR	<code>gt_adr.py merge <project></code>

No session writes a shared file, so there is nothing to claim and nothing to guard, and `git add` scopes to your own work by construction rather than by discipline.

ADR numbers are the harder half, because a number is a scarce identity — two sessions may both write a log line in the same instant, but they cannot both be ADR-6. A counter that is read and then written back does not fix that: read-then-write is two operations, so both sessions read 5, both write 6, and both use ADR-6. `gt_adr.py allocate` instead creates the slot with a single atomic exclusive create, so the filesystem decides the winner. Verified under contention: 100 racing process pairs, 200 allocations, no duplicates.

Migration (`gt_log.py migrate` , `gt_adr.py migrate <project>`) freezes the existing file as a baseline that sorts first — no history is rewritten and nothing is renumbered — and refuses unless the merge reproduces the original byte for byte. It also refuses on a project whose ADR numbers already collide, because renumbering would invalidate inbound references; `gt-lint 's adr-collision` check reports those for an owner to resolve.

One limit, stated plainly: an exclusive create is atomic on one filesystem. Two machines writing one project through a synced folder can both take a slot, and the sync resolves it as a conflicted copy. The lint check is what catches that; this is not distributed consensus.

Installed, and actually wired (0.9.13)

The session-start component check answers two questions, not one:

Question	How
Are the right files here, unmodified?	every shipped file is hashed into <code>MANIFEST.json</code> at install time and compared
Is any of it connected to anything?	<code>MANIFEST.json</code> also declares the nine hook entries the plugin expects in <code>settings.json</code> , and the check compares those too

Before 0.9.13 only the first question was asked, so a machine with every file installed and an empty `settings.json` reported **components: clean** while nothing ran. The clean line now reads *"installed matches , all 9 hooks wired"* — the second clause is the part that was missing.

Two states are reported: `unwired` (no entry for that event names the script — it never runs) and `badpath` (wired, but the command names a path that does not exist on this machine, which is what a deleted version directory or an unquoted path containing a space produces).

The declaration lives in `gt_components.HOOK_REGISTRATIONS` and is what `install.sh` registers *from*, so the installer and the checker cannot disagree about what "wired" means. `install.sh` verifies its own six entries immediately after writing them; the three enforcement hooks are owned by `vault_init.py install-core-rules` , which needs a vault. `selftest.sh` asserts all nine from outside — the only vantage point that still works when nothing is wired at all.

gt-wiki Skills (5)

The gt-wiki plugin provides an LLM-powered knowledge base separate from the Golden Thread memory hierarchy. Sources are ingested immutably; Knowledge pages are synthesized summaries linked bidirectionally.

Key design decision: The vault `CLAUDE.md` is kept slim (architecture + pointer only). Page format schema lives in `Knowledge/_template.md` and is read on demand by the ingest and refresh skills — it is not loaded into every session's context.

Command	What it does
<code>/gt:gt-wiki-init</code>	Set up a new wiki vault from scratch. Runs <code>vault_init.py</code> deterministically — creates <code>Sources/</code> , <code>Knowledge/</code> , <code>index.md</code> , <code>log.md</code> , seeds <code>CLAUDE.md</code> from template, and stamps <code>Knowledge/_template.md</code> .
<code>/gt:gt-wiki</code>	Query the wiki. Reads <code>index.md</code> first, follows wikilinks, falls back to grep, deep-digs Sources for precision. Logs every query.
<code>/gt:gt-wiki-ingest</code>	Add a source: fetch or paste, store immutably in <code>Sources/</code> , discuss with user, write Knowledge pages per <code>_template.md</code> , cross-link bidirectionally, then update <code>index.md</code> and <code>log.md</code> through <code>wiki_log.py</code> so every entry has the same shape. Records <code>upstream_sha:</code> for sources inside a git repo.
<code>/gt:gt-wiki-lint</code>	Run <code>wiki_lint.py</code> (10 deterministic checks). Interprets findings, proposes fixes, records declines in <code>lint-declines.md</code> so nothing gets re-litigated.
<code>/gt:gt-wiki-refresh</code>	Check selected sources for upstream changes. <code>wiki_refresh.py</code> detects changes deterministically for local sources (<code>git fetch</code> + <code>git diff</code> from the source's <code>upstream_sha:</code> , or its <code>ingested:</code> date) and flags web-only sources for the LLM to fetch and compare. Supersedes changed ones with new immutable source files. Updates citing Knowledge pages.

Vault Structure

```
<vault>/
  CLAUDE.md           ← architecture + pointer to Knowledge/_template.md
  index.md            ← navigational index of all Knowledge pages
  log.md              ← audit trail: every create, ingest, promote, retire
  review-queue.md     ← items flagged for owner review (written by gt-lint)

  Sources/            ← IMMUTABLE raw originals
    YYYY-MM-DD <title>.md

  Knowledge/          ← cross-project wiki pages
    _template.md      ← page format schema (read by ingest/refresh skills)
    <Page Title>.md
```

```

global-memory/
  MEMORY.md          ← index; loaded every session
  <topic>.md

Projects/
  README.md          ← master project list
  CONVENTIONS.md     ← lifecycle phases, file roles, tag taxonomy
  PROTOCOL.md        ← cross-project process rules
  INFRASTRUCTURE.md  ← server fleet, defined once

<project-slug>/
  README.md          ← status board + YAML frontmatter
  source.md          ← where code lives, topology, deploy file plan
  idea.md            ← original brain dump – IMMUTABLE
  research.md        ← append-only findings
  decisions.md       ← append-only ADRs
  design.md          ← current architecture
  spec.md            ← handoff artifact (created when design is complete)
  runbook.md         ← operational procedures (optional)
  memory/
    MEMORY.md        ← session memory index
    <topic>.md

golden-thread/
  core-rules/        ← canonical Core rule definitions

```

Immutability Model

File	Mutability	Rule
Sources/*.md	Immutable	Never edited; superseded by new files
idea.md	Immutable	Origin story; never changed
research.md	Append-only	History is extended, never rewritten
decisions.md	Append-only	Reversed by adding a new ADR, never by editing
design.md	Mutable	Always describes the current architecture

File	Mutability	Rule
<code>spec.md</code>	Mutable (scope only)	Self-contained handoff doc
<code>runbook.md</code>	Mutable	Operational HOW-TO; incubator before CLAUDE.md
<code>PROTOCOL.md</code>	Append-only	Cross-project process rules
<code>Knowledge/*.md</code>	Mutable	Synthesized summaries; cite <code>Sources/</code>
<code>core-rules/*.md</code>	Mutable	Editing here changes what hooks inject

Context Footprint

What loads	When	Approx tokens
<code>~/.claude/CLAUDE.md</code> (global instructions)	Every session	~200
<code><vault>/CLAUDE.md</code> (architecture + pointer)	Every session in vault dir	~100
<code>global-memory/MEMORY.md</code> (index only)	Every session	~50
<code>Knowledge/_template.md</code> (page schema)	Only during ingest/refresh	~150
Project memory files	On demand via <code>gt-open</code>	Varies

The slim CLAUDE.md design (introduced in gt-wiki 0.1.0) drops the vault's cold-start footprint from ~1,300 tokens to ~350.

Source Topology

Every project records where its code lives in `source.md` .

Topology	Meaning
<code>local</code>	A folder on this machine. No deploy step.
<code>remote</code>	One repo, one server; server pulls from GitHub.
<code>bastion-jump</code>	Several servers, all reached through one gateway host.
<code>bastion-direct</code>	Several servers, each reachable independently.

Bastion projects carry a **file plan** marking each deployed file `static` (byte-identical everywhere) or `unique` (per-server variant that must never be cross-deployed).

Install

```
# From zip
unzip golden-thread-plugin.zip
cd golden-thread-plugin
bash install.sh
# Restart Claude Code
```

Installs both `gt` (v0.12.3) and `gt-wiki` (v0.1.2) as separate plugins under the `golden-thread-plugin` marketplace. Requires Python 3.8+.

`install.sh` installs the **newest version directory** present, not a hardcoded constant — pass an argument only to roll back deliberately (`./install.sh 0.9.3`). Never pipe it to `head` : `set -o pipefail` turns the closed pipe into an abort partway through, leaving the cache updated and registration undone.

Typical Session Pattern

```
/gt:gt-open my-project          # load project, summarize state

# ... work happens ...
```

```
/gt:gt-route          # "where is this going?" – run it when the session
                      # has drifted, or before writing something down and
                      # you are unsure which file it belongs in

/gt:gt-work           # write back findings, update docs

# periodically

/gt:gt-lint           # catch structural drift
/gt:gt-wiki-ingest <url> # add a source to the wiki
/gt:gt-promote        # graduate a finding to Knowledge or global-memory
```

Requirements

- Python 3.8+
- Claude Code (any version)
- Obsidian (optional — the vault is plain markdown; Obsidian is the GUI layer)